

Angular Auth Guards

In layout.component.htmls

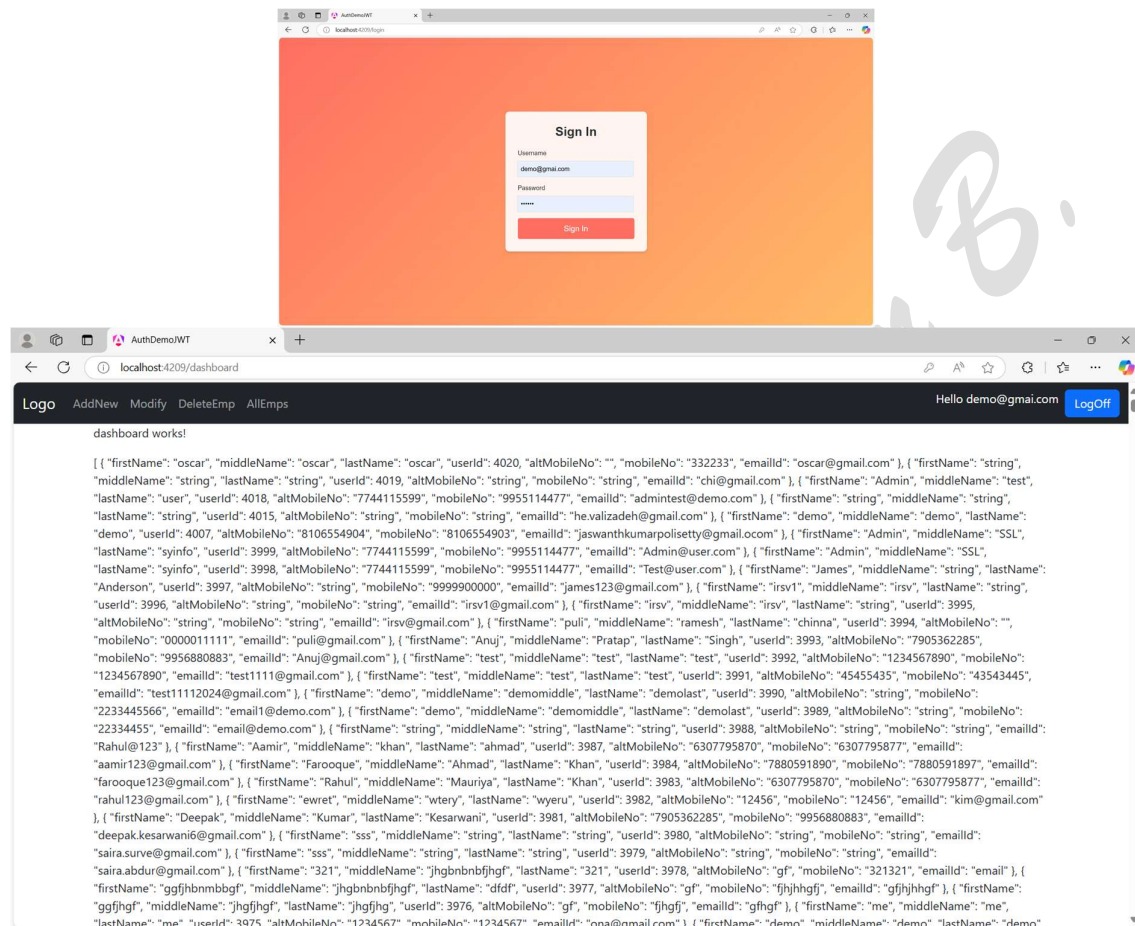
In `layout.component.ts` modify:

Presented by
Ranjan Bhatnagar

Add to onLogin method of login.component.ts:

```
localStorage.setItem('loginUser', this.userObj.EmailId)
```

Let's test once, is it working fine or not?



Looks fine. But there is one loophole, if you logoff it will divert to Login Page and also, we cleared logged-in user data also from local storage. Now if you directly use URL for Dashboard it will display user but not the logged-in user as it is not logged in page.

Here comes the role of Guards concept of Angular.

What are Guards?

In Angular, guards are scripts that control access to different parts of an application by deciding whether to allow or deny a user's navigation to a specific route. Guards are a crucial part of Angular's routing mechanism and are used to implement security and control navigation flow.

Here are some types of guards in Angular 18:

- CanActivate: Controls whether a route can be activated
- CanActivateChild: Controls whether child routes of a particular route can be activated

Started updating as on July 18th, 2024

- CanDeactivate: Controls whether a user can navigate away from the current route
- CanLoad: Controls whether a module can be lazy-loaded
- Resolve: Pre-fetches data before a route is activated

You can use route guards to prevent users from navigating to parts of an application without authorization. To create a route guard use command:

...authDemoJWT\src\app\guards> ng g guard auth

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POLYGLOT NOTEBOOK

PS D:\NGLearn\Auth-11-11-2024\authDemoJWT\src\app\guards> ng g guard auth
? Which type of guard would you like to create? (Press <space> to select, <a> to toggle all, <i> to invert selection, and <enter> to proceed)
>) CanActivate
  () CanActivateChild
  () CanDeactivate
  () CanMatch
```

Select CanActivate by space bar and press enter, it will create a guard which is an arrow function:

```
import { CanActivateFn } from '@angular/router';

export const authGuard: CanActivateFn = (route, state) => {
  return true;
};
```

In this function we will be checking that user is logged in or not.

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';

export const authGuard: CanActivateFn = (route, state) => {
  const router = inject(Router);
  if (localStorage.getItem("loginUser") !== null) {
    return true;
  }
  else {
    router.navigateByUrl("login");
    return false;
  }
};
```

Now apply this guard to the routes you want to protect:

```
children: [
  {
    path: 'dashboard',
```

```
component: DashboardComponent,  
canActivate: [authGuard]  
}  
]
```

❄❄End of Chapter❄❄